

Representing text and integers

(reference: text Sections 2.4 and 3.2, pages 88-89)

everything (data, instructions) represented in binary
(1s and 0s)

text representations

- **ASCII** (each character allocated 1 byte)
- with **Unicode**, able to represent characters from all languages as well as many other useful symbols; character encodings based on Unicode include the variable length encodings UTF-8, UTF-16, and others
- represent strings by concatenating representations of characters
- could use delimiter to indicate end of string, e.g. byte containing 0 as with “.asciz” (and in C), or could store a count of the number of characters in the string (as in Java)

representations of unsigned integers

- **BCD** (“Binary-coded Decimal”): use 4 bits to represent each decimal digit

- **base 2 (binary)**

e.g. 101010_2 represents:

$$1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 = 42_{10}$$

binary representation makes more efficient use of (binary!) storage, enables more efficient hardware arithmetic implementations

octal (base 8) and **hexadecimal (base 16)** numbers are more compact and easily readable by people than binary numbers, and yet can easily convert to/from binary

$$553_8 = 5 \times 64 + 5 \times 8 + 3 = 363_{10}$$

$$16B_{16} = 1 \times 256 + 6 \times 16 + 11 = 363_{10}$$

note: $16B_{16}$ could also be written as $16B_{\text{hex}}$ or $0x16B$

conversions to/from binary easy since **1 octal digit corresponds exactly to 3 binary digits** (since $2^3 = 8$), **1 hex digit corresponds exactly to 4 binary digits** (since $2^4 = 16$) *let's try it* $553_8 = ??_2 = ??_{16}$

conversions between numbers in different bases, in cases when neither of the bases is an integer power of the other

base n to base 10: just apply definition of base n !!

$$24B_{16} = ?? \times 16^{??} + ?? \times 16^{??} + ??$$

base 10 to base n : methods include “removing powers”, and “repeated division”

- example of “removing powers” method

$$53_{10} = ??_2$$

$$53_{10} = ?? \times 2^{??} + ?? \times 2^{??} + \dots = ??_2$$

- examples of “repeated division” method

$$587_{10} = ??_{16}$$

(repeatedly divide by 16, remainders give digits of answer from right to left; $587/16 = 36$ with remainder 11, so rightmost digit is B; $36/16 = 2$ with remainder 4, so next digit is 4, $2/16 = 0$ with remainder 2, so final digit is 2; $587_{10} = 24B_{16}$)

base n to base m when neither n nor m is 10 (and neither of the bases is an integer power of the other): could first convert from base n to base 10 and then from base 10 to base m

$$4A_{13} = ??_{11}$$

addition, subtraction of numbers in different bases

carrying, borrowing, work similarly as with base 10 numbers (e.g. how does one calculate $753 + 491$, or $753 - 491$, by hand?)

$$\begin{array}{r} 1111_2 \\ + \underline{1101}_2 \\ \hline 11100_2 \end{array} \qquad \begin{array}{r} 1011001_2 \\ - \underline{101100}_2 \\ \hline 101101_2 \end{array}$$

finite representation

with n bits, can represent unsigned integers (as base 2 numbers) from 0 to $2^n - 1$